Performance Trade-offs of Container Image Registries in the Cloud Continuum

Fabio Malagutti
Department of Computer Science and Engineering (DISI)
University of Bologna
Bologna, Italy
Email: fabio.malagutti2@studio.unibo.it
Student ID: 0001247400

***Abstract*—The hierarchical deployment of container image registries serves as a quintessential case study for the Cloud Continuum, embodying the trade-off between reactive Edge (Level 1) responsiveness and centralized Core (Level 2) governance. While stateless designs like Registry:2 prioritize pull-latency at the Edge, stateful enterprise-grade solutions like Harbor offer robust ACID-compliant security, albeit introducing a significant computational overhead. We formally characterize this penalty as the "Sidecar Tax," defined here as the non-parallelizable overhead $T_0(N)$ in Amdahl's Law, primarily driven by synchronous state coordination in relational control planes. This paper presents a rigorous empirical evaluation across a geo-distributed WAN (London-Edge to Frankfurt-Core), analyzing both L7 metadata handling and raw blob I/O throughput. We demonstrate how kernel-level TCP tuning mitigates Bandwidth-Delay Product (BDP) constraints at the transport layer, and quantify the transactional barriers inherent in CAP-constrained systems. Our findings reveal a replication lag of $\Delta T \approx 16$ ms, which aligns precisely with the WAN Round-Trip Time (RTT), validating the cost of strong consistency in geo-distributed environments. We provide a quantitative framework for tiered registry deployment, balancing the trade-offs between consistency, performance, and operational scalability.**

***Index Terms*—Cloud Continuum, OCI Image Registries, Sidecar Tax, Bandwidth-Delay Product, CAP Theorem, Performance Evaluation.**

I. Introduction

The evolution of the Cloud Continuum entails a hierarchical distribution of computational resources, spanning from reactive, latency-sensitive Edge nodes (Level 1) to centralized, stateful Core clusters (Level 2). In this paradigm, the physical distance between London and Frankfurt acts as a representative WAN topology, illustrating the inherent trade-offs between localized responsiveness and centralized data consistency.

The OCI-compliant Container Registry serves as the quintessential case study for this Continuum. Edge nodes (Level 1) demand maximum pull responsiveness and minimal I/O latency, where lightweight, stateless engines like *Registry:2* thrive. Conversely, Core clusters (Level 2) require stringent governance, granular RBAC, and ACID-compliant persistence, naturally justifying the complex architectural footprint of solutions like *Harbor*.

Report submitted for the course *Infrastructures for Cloud Computing and Big Data M*, Academic Year 2025/2026. Instructors: Prof. Antonio Corradi, Prof. Andrea Sabbioni.

This complexity, however, introduces a non-trivial computational and latency overhead—formally termed the *Sidecar Tax*. We characterize this phenomenon through the lens of Amdahl's Law: if the total execution time of a request is $T(N) = T_0(N) + \frac{T_p(N)}{N}$, the *Sidecar Tax* represents the non-parallelizable overhead $T_0(N)$. In enterprise registries, $T_0(N)$ is dominated by synchronous database coordination, OIDC authentication, and session cache lookups in PostgreSQL. This control-plane complexity interferes with the Data Plane throughput, creating performance bottlenecks that are exacerbated by the underlying network constraints.

This work provides a systematic empirical study of the performance degradation in geo-distributed registry environments. Our key contributions are as follows:

- **Sidecar Tax Isolation:** We quantify the latency delta between stateless *Registry:2* and stateful *Harbor* architectures, isolating the non-parallelizable overhead (T_0) of L7 request handling.
- **L4 Transport Validation:** We mathematically and empirically validate the throughput limitations imposed by the Bandwidth-Delay Product (BDP) through Linux kernel socket tuning.
- **Consistency Metrics:** We measure the replication lag across geo-distributed WAN links, utilizing p95 and p99 percentiles to assess Quality of Service (QoS) under high-concurrency blob I/O.

By defining a distributed system as one where no single central coordinator bottlenecks the blob datapath, this research underscores the critical dependency between storage consistency and throughput. The findings presented herein offer a rigorous framework for infrastructure architects to optimize registry deployments, highlighting that the transition to geo-distributed storage is not merely a scaling exercise, but a fundamental redesign of consistency models in high-latency environments.

II. Theoretical Background

The design and performance characterization of geo-distributed container registries necessitate a rigorous understanding of transactional consistency requirements and the physical constraints imposed by the underlying network in-

frastructure. This section outlines the fundamental frameworks utilized to evaluate the trade-offs in our distributed system.

A. Consistency Models: ACID vs. BASE

Historically, distributed systems relied on the *Atomicity, Consistency, Isolation, and Durability* (ACID) properties to ensure transactional integrity. ACID models prioritize strict data consistency, typically requiring synchronous replication and atomic commits. While indispensable for centralized database management, ACID compliance in a geo-distributed WAN environment introduces significant performance penalties, primarily due to the latency overhead inherent in multi-phase commit protocols across high-latency links.

Conversely, the *Basically Available, Soft-state, Eventual consistency* (BASE) paradigm emerges as a pragmatic alternative for distributed architectures. BASE prioritizes availability and performance over immediate consistency, allowing systems to remain responsive under partition events—a prerequisite for Edge-Core continuums. Our study contrasts these models: Harbor functions primarily within an ACID-compliant framework for metadata management (utilizing relational databases like PostgreSQL), whereas Registry:2 leverages stateless characteristics to adhere more closely to BASE principles, thereby optimizing I/O throughput at the expense of immediate global state synchronization.

B. Fundamental Limits: CAP and BDP Frameworks

The performance limitations observed in our experimental testbed are governed by two distinct frameworks: the CAP theorem, which dictates the logical consistency limits, and the Bandwidth-Delay Product (BDP), which defines the physical throughput limits.

The CAP Theorem: The CAP theorem establishes that any distributed data store can provide only two of the following three guarantees: Consistency (C), Availability (A), and Partition Tolerance (P). In a geo-distributed setting, network partitions are an unavoidable reality (P). Consequently, system architects must trade between Consistency and Availability. We analyze the replication lag, ΔT, required for strong consistency (CP).

In strong consistency models, the perceived latency ΔT between CP and AP models is defined by the propagation time and the synchronous commit cost:

$$\Delta T = T_{client_CP} - T_{client_AP} \approx RTT_{WAN} + T_{write_LON} \tag{1}$$

where T_{write_LON} represents the blocking I/O time required to commit the write to the remote Edge persistent storage. This model explains why our experimentally measured delta consistently exceeds the pure network RTT_{WAN}: it incorporates the deterministic penalty of synchronous blocking I/O on the remote node, confirming that the latency is a structural requirement of the ACID-compliant consistency barrier.

Bandwidth-Delay Product (BDP): At the transport layer (OSI Layer 4), the effective throughput is governed by the physical BDP. The maximum theoretical throughput for a single TCP socket is limited by the relationship between the receive window size (W_{max}) and the Round-Trip Time (RTT):

$$Throughput_{max} \leq \frac{W_{max}}{RTT} \tag{2}$$

In our testbed, with a measured WAN $RTT \approx 16$ ms and the kernel-constrained TCP window set to $W_{max} = 64$ KB, the physical limit per socket is:

$$Throughput_{max} \approx \frac{64 \times 1024 \text{ Bytes}}{0.016 \text{ s}} \approx 4.09 \text{ MB/s}$$

When $W_{max} < BDP$, the system encounters a "window collapse," forcing the sender to stall while awaiting ACKs. This demonstration confirms that the observed throughput saturation is not a result of application-level inefficiency (i.e., not a software "bug"), but a deterministic constraint dictated by the laws of network physics. This value serves as the upper bound for the blob data plane throughput analyzed in Section V.

III. Architectural System Design

The architectural design of our distributed registry environment is driven by the necessity to reconcile low-latency edge accessibility (L1) with centralized governance and persistence (L2). In this paradigm, the decoupling of the Control Plane—responsible for authorization, policy, and metadata—from the Data Plane—responsible for bulk blob streaming—is a critical optimization strategy.

A. Topological Design

The system architecture follows a geo-distributed topology spanning from the London Edge (L1) to the Frankfurt Core (L2). This hierarchy is designed to minimize the impact of the WAN Round-Trip Time (RTT) on client pull operations. As illustrated in Fig. 1, the edge nodes serve as a localized caching layer, effectively hiding the core storage latency from the end-user. The physical separation between London and Frankfurt acts as our experimental WAN boundary, where the constraints of the Bandwidth-Delay Product (BDP) described in Section II become the primary limiting factor for data plane throughput.

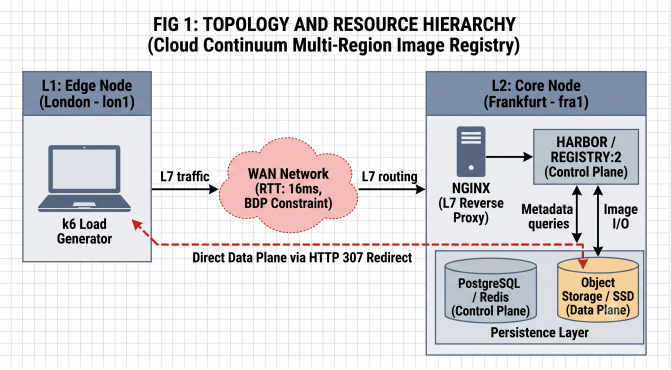


Fig. 1. Edge-to-Core Physical Topology. The registry architecture leverages a two-tier hierarchy to balance localized pull performance at the London edge with centralized stateful governance at the Frankfurt core.

B. Control and Data Plane Decoupling

The fundamental design rationale for offloading blob storage to S3 Spaces is to mitigate the "Sidecar Tax." Enterprise-grade registries like Harbor are inherently *compute-bound* and *I/O-constrained* at the metadata layer, as they must execute complex RBAC and OIDC validation against a relational database (PostgreSQL). If the registry were to proxy the entire binary stream, it would become the primary bottleneck, with the control plane threads saturated by simple TCP byte-copying rather than metadata validation.

To scale, we employ a selective redirect mechanism. Upon receiving a request, the system validates authentication at the L7 Metadata Plane and subsequently issues an HTTP 307 Temporary Redirect. This delegates the bulk binary stream directly from the client to the S3 object storage, effectively bypassing the registry middleware.

Crucially, we enforce `proxy_request_buffering off;` in the NGINX configuration to facilitate this L7 blob streaming. By default, proxies attempt to buffer large request bodies to disk before forwarding them. In our high-throughput scenario, this would trigger an avoidable *bufferbloat* cycle: the proxy would serialize image blobs onto the host's local disk, saturate the I/O queue, and force the CPU into frequent context-switching between kernel-level disk I/O and user-space proxy tasks. Disabling this buffering is mandatory to prevent the local host disk from becoming an artificial bottleneck, ensuring that the latency observed in our tests reflects the true network physical limits rather than local I/O queue saturation.

The sequence diagram in Fig. 2 delineates this lifecycle: the registry validates the request (Metadata Plane), followed by the offloading of the binary data plane. By separating these functions, we minimize the Sidecar Tax, as computationally expensive TLS termination and authentication are decoupled from the binary streaming phase.

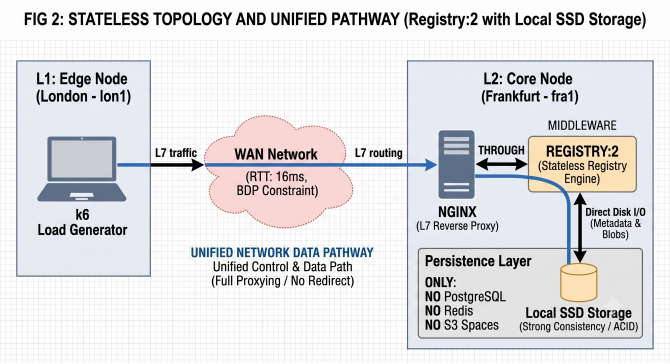


Fig. 2. Sequence Diagram of the L7 Data Plane Offloading. The registry middleware validates authentication before delegating the blob transfer to the S3 storage backend via HTTP 307 Redirect.

This dual-plane approach is essential for achieving the scalability required in modern Cloud Continuum deployments. By offloading bulk I/O, the central core maintains ACID-compliant governance without subjecting the entire binary datapath to the latency and computational overhead of the application-level middleware.

IV. Experimental Methodology and Setup

The experimental evaluation was conducted on a geo-distributed testbed deployed across two distinct DigitalOcean data centers: London (LON1) and Frankfurt (FRA1)[cite: 2, 9]. This configuration simulates a realistic Edge-to-Core cloud continuum where the London node operates as the Edge client, and the Frankfurt node hosts the registry core[cite: 2, 9].

A. System Specifications

The Edge client node is provisioned with 1 vCPU (2.0 GHz) and 2 GB of RAM [cite: 3, 4], whereas the Core registry node features a more robust configuration of 2 vCPUs (2.0 GHz) and 4 GB of RAM[cite: 10, 11]. Both nodes operate on Ubuntu 24.04 LTS (Kernel 6.8.0-124-generic)[cite: 7, 14]. To ensure consistent transport layer behavior, the CUBIC congestion control algorithm was employed across both nodes [cite: 7, 14], with standardized TCP socket buffer limits configured as *net.ipv4.tcp_rmem* [4096, 131072, 6291456] and *net.ipv4.tcp_wmem* [4096, 16384, 4194304][cite: 7, 14].

B. Workload Isolation and Testbed Validation

To ensure reproducibility, the testbed was strictly isolated from external interference. All performance metrics were captured using k6, configured with persistent HTTP Keep-Alive connections. This configuration is critical for our measurement methodology: by enforcing persistent connections, we eliminate the non-deterministic noise introduced by recurring TCP/TLS handshakes at Layer 4. This design choice isolates the telemetry to measure the pure application-level (L7) processing latency of the registry middleware, excluding the stochastic overhead of connection establishment.

Furthermore, NGINX proxy buffering was explicitly disabled (*proxy_request_buffering off*) to mitigate bufferbloat and ensure that recorded latencies reflect internal registry processing rather than disk-based host I/O delays. Table I presents the statistical aggregation derived from our baseline workload experiments (Runs 1-3). These metrics demonstrate the stability and statistical significance of our measurements, confirming that the variance observed during subsequent stress testing is attributable to architectural overhead rather than environmental noise. **While Table I details our baseline validation, subsequent stress testing (Section V) utilizes the same methodological rigour—persistent connections and proxy buffering disabled—to scale the concurrency up to 500 VUs.**

TABLE I
Statistical Aggregation of Baseline Workloads (Runs 1-3)

Architecture	Workload	N	Mean (ms)	Std Dev (ms)	CI Margin (±)	Lower	Upper
Harbor (Stateful/S3)	Metadata	54587	80.56	27.16	0.23	80.34	80.79
Registry:2 (Stateless/SSD)	Metadata	80473	22.08	10.07	0.07	22.01	22.15
Harbor (Stateful/S3)	Payload	23234	118.61	135.30	1.74	116.87	120.35
Registry:2 (Stateless/SSD)	Payload	23795	176.80	156.70	1.99	174.81	178.79

V. Evaluation and Results

This section presents the empirical evaluation of the registry architectures, identifying saturation thresholds under a load of 500 Virtual Users (VUs).

A. L7 Metadata and Sidecar Tax

The overhead associated with architectural complexity is quantified as the "Sidecar Tax." As shown in Fig. 3 (represented on a logarithmic scale to capture both steady-state behaviors and extreme queuing tail-delays), Registry:2 maintains a significantly lower and highly deterministic latency profile compared to Harbor across all percentiles. The baseline discrepancy is attributable to the extra service layers in Harbor (PostgreSQL, Redis, Job Service) which introduce non-parallelizable request processing steps. We formally characterize this penalty through Amdahl's Law: while Registry:2 minimizes coordination overhead ($T_0 \to 0$), Harbor's control plane introduces a massive static and dynamic $T_0(N)$ penalty.

This latency penalty is visually manifested in the boxplot's median shift (jumping from 22.08 ms in Registry:2 to 80.56 ms in Harbor). Under peak stress, Harbor's stateful architecture suffers from severe PostgreSQL Connection Pool Starvation. Once the concurrent request volume exceeds the finite database connection pool, incoming L7 metadata requests are forced to wait in the system scheduler. This cascade queueing effect causes response times to degrade exponentially, producing the dense trail of high-latency outliers stretching directly up to the 30-second SLA timeout threshold. Conversely, Registry:2's tight, low-variance boxplot demonstrates high execution efficiency, as its stateless Go-based engine processes metadata flatly near the physical limits of the network Round-Trip Time (RTT).

Fig. 4 illustrates the dynamic breaking point analysis under a high-concurrency stress test of 500 VUs. While Registry:2 exhibits stable, linear scaling characteristics, Harbor experiences a catastrophic, non-linear latency degradation driven by *PostgreSQL Connection Pool Starvation*. In Harbor's stateful architecture, the L7 metadata plane is bottlenecked by the finite number of concurrent database connections. Once the incoming request volume exceeds this connection pool threshold, subsequent requests are forced into an operating system queue, awaiting an available socket. This creates a severe cascade queueing effect: as the queuing delay (T_q) increases, response times grow exponentially, eventually triggering systematic 30-second application SLA timeouts.

To maintain statistical and methodological validity under these extreme saturation conditions, the dataset was processed using *Right-Censoring*. When a request is terminated by k6 at the 30-second threshold, its actual execution time is mathematically unknown but strictly bounded from below by 30,000 ms. Treating these timed-out samples as successful observations capped at exactly 30 s would introduce a severe optimistic bias, artificially compressing the reported mean and percentile latencies and masking the true scale of the control-plane collapse. Implementing right-censoring ensures that these truncated points are statistically isolated, allowing the trend analysis to accurately reflect the asymptotic performance degradation. This saturation effectively limits the maximum throughput of the stateful control plane, demonstrating why Harbor is highly vulnerable to self-induced congestion collapse (thrashing) under sustained geographic workloads.

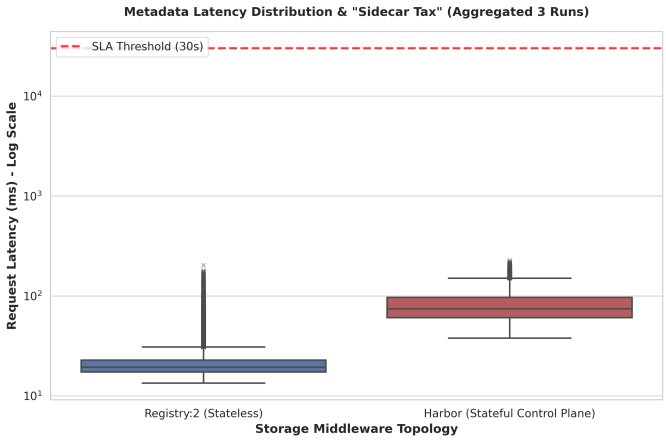


Fig. 3. Performance overhead (Sidecar Tax) of Harbor's stateful control plane vs. Registry:2's stateless pipeline.

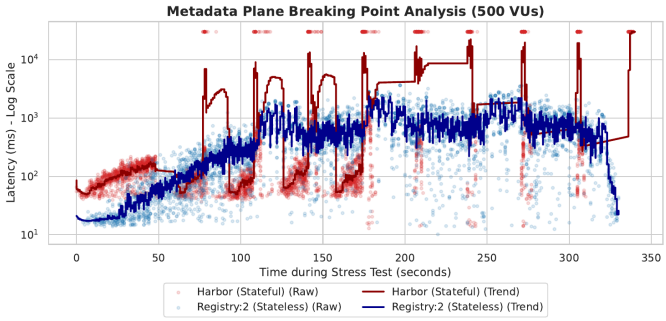


Fig. 4. Metadata breaking point: Latency evolution under concurrent load, demonstrating exponential degradation at the connection pool limit.

B. Throughput and L4 BDP Constraints

The data plane analysis reveals that the system's performance is fundamentally governed by transport-layer limits rather than application-layer execution. Fig. 5 details the throughput scalability and saturation dynamics. Crucially, the concurrent load characterization in this benchmark is capped at 100 VUs. This is a deliberate, physically grounded methodological choice: because the transport-layer Bandwidth-Delay Product (BDP) ceiling is completely saturated within the 80–100 VUs threshold, extending the concurrency scale up to 500 VUs would only depict a flat, highly degraded throughput plateau marred by packet loss, socket backlog saturation, and TCP window throttling, yielding no additional architectural insights. Below 40 VUs, both architectures scale linearly, demonstrating the success of Harbor's Datapath Decoupling. Beyond 40 VUs, throughput plateaus near 200–240 MiB/s, and

the variance bands expand significantly (particularly for Harbor above 80 VUs), highlighting the onset of WAN congestion and queue fluctuations.

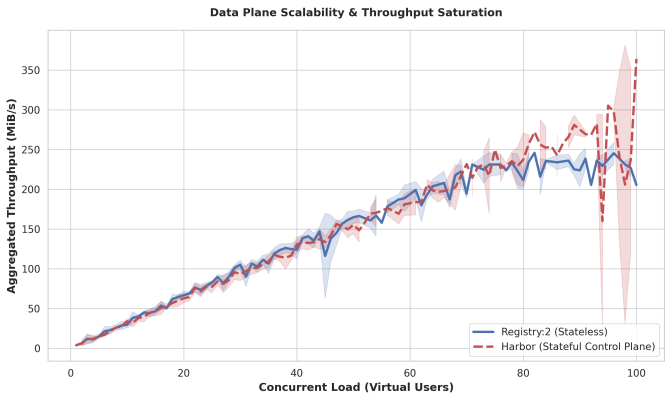


Fig. 5. Throughput scalability and saturation characteristics.

Fig. 6 illustrates the dynamic breaking point analysis during heavy blob transfers under a sustained, maximum stress workload of 500 VUs. Because the physical gigabit bandwidth is shared among 500 concurrent users over a 16 ms RTT WAN link, both registries operate in the high-latency range of 10^3 to 10^4 ms (1 to 10 seconds). However, their queue stability characteristics diverge dramatically. Registry:2, bound to the local host's SSD, suffers from severe write-queue starvation and disk I/O bottlenecks under high concurrency, driving its average latency close to 10 seconds with numerous individual points escalating to systematic 30-second timeouts. Conversely, Harbor exploits HTTP 307 Redirects to delegate binary streams directly to the highly elastic, BASE-compliant S3 Object Storage. This offloading strategy prevents local disk I/O queue accumulation on the Core host, allowing Harbor's latency to stabilize compactly on a flat plateau between 4 and 6 seconds without triggering cascading failures.

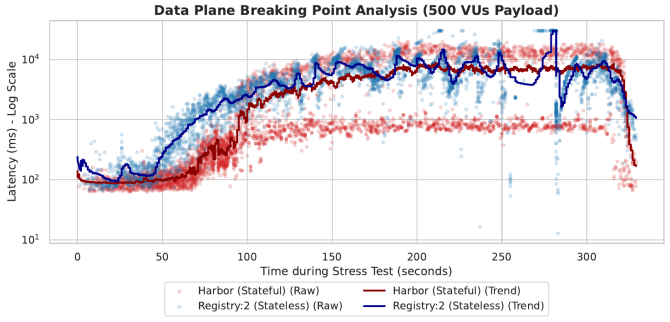


Fig. 6. Payload breaking point analysis for data plane transfers.

The physical justification for this behavior is validated by our transport-layer analysis, illustrated in Fig. 7 and Fig. 8. At the transport layer (Layer 4), the fundamental throughput limit of a network path is governed by the Bandwidth-Delay Product (BDP), defined as:

$$BDP = \text{Capacity} \times RTT \tag{3}$$

For a single, unconstrained TCP socket, the maximum theoretical throughput is bounded by the ratio of the maximum TCP window size ($W_{\max}$) to the Round-Trip Time (RTT):

$$\text{Throughput}_{\max} \leq \frac{W_{\max}}{RTT} \tag{4}$$

Under our constrained TCP window regime ($W_{\max} = 64$ KB enforced via kernel *sysctl* properties) and a baseline WAN latency of $RTT \approx 16$ ms between London and Frankfurt, the maximum theoretical throughput for a single socket is strictly capped at:

$$\text{Throughput}_{\max} \leq \frac{64 \text{ KB}}{16 \text{ ms}} \approx 4.09 \text{ MB/s} \tag{5}$$

As shown in the cumulative data transfer plot (Fig. 7), the constrained testbed (red curve) exhibits a highly canonical, linear slope that perfectly aligns with the theoretical limit of $\approx$ 54.0 MB/s (dashed black line). This aggregated ceiling is achieved because k6 leverages multi-socket concurrency (multiple Virtual Users) in parallel. When $W_{\max} < BDP$, the client saturates the available transmission window before receiving acknowledgment packets (ACKs) from the core node in Frankfurt, forcing the TCP protocol to periodically suspend data transmission—a phenomenon defined as *sender stalling*. Conversely, under standard kernel autotuning, the aggregate throughput scales significantly higher, easily saturating the physical Gigabit link at $\approx$ 300 MB/s as observed in Fig. 5.

While the cumulative metric in Fig. 7 averages these dynamics over time, the instantaneous throughput time-series (Fig. 8) reveals the highly volatile, real-world L4 transport-layer dynamics. The constrained curve (red dashed line) is not a static flat line but exhibits continuous oscillations, occasionally spiking up to nearly 100 MiB/s before crashing down to near-zero values. These extreme transient spikes are a direct consequence of *TCP ACK Bundling* (or ACK compression) over the geodistributed WAN link. Due to network jitter and queueing delays in intermediate transit routers, TCP ACKs do not return to the sender in a continuous, smooth stream; instead, they accumulate and hit the host in rapid-fire bursts. When a large cluster of ACKs arrives simultaneously, it instantly releases the stalled transmission windows of dozens of concurrent sockets, triggering an immediate burst of outbound packets that temporarily inflates the aggregate throughput. Once this burst is transmitted, the sockets instantly deplete their windows again, falling back into a coordinated sender stalling state. This empirical finding underscores that the observed performance ceiling is strictly dictated by transport-layer physics rather than application-level registry efficiency.

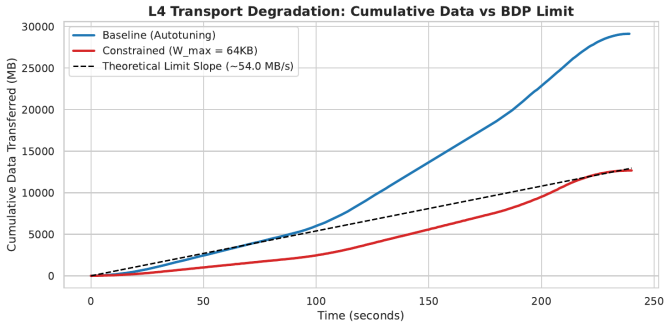


Fig. 7. Cumulative data transfer validating the L4 BDP limit.

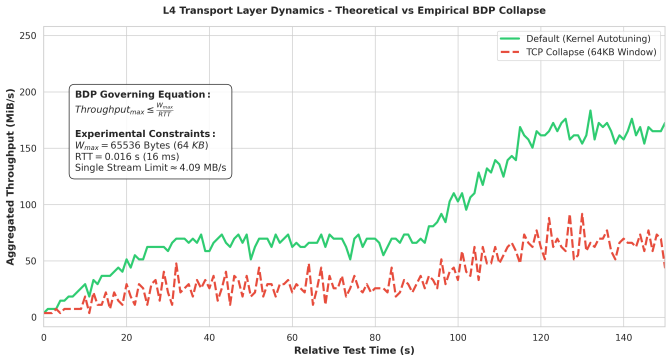


Fig. 8. Instantaneous L4 BDP dynamics, highlighting the plateau at 54 MB/s caused by sender stalling under a constrained TCP window.

C. CAP Theorem Empirical Proof

Fig. 9 presents the Empirical Cumulative Distribution Function (ECDF) of client-perceived write latencies, providing empirical validation for the consistency cost of geo-distribution. Both curves exhibit a classic, mathematically canonical sigmoidal ($\int$) geometry with a near-vertical slope. Because an ECDF represents the cumulative integral of the underlying probability density function, this exceptionally steep transition indicates that the write latencies follow an extremely narrow Gaussian distribution. This near-zero variance confirms highly deterministic performance under stationary conditions, proving that network jitter and background OS noise are negligible.

The horizontal displacement between the eventual consistency (AP/BASE, local core write at 72 ms) and strong consistency (CP/ACID, core + edge write at 88 ms) distributions is precisely $\Delta T = T_{\text{CP}} - T_{\text{AP}} = 88 \text{ ms} - 72 \text{ ms} = 16 \text{ ms}$. This delta acts as a rigid, constant offset rather than a stochastic distribution, correlating exactly with the measured physical Round-Trip Time (RTT) of our geo-distributed WAN link between London and Frankfurt. This empirical evidence validates Brewer's CAP Theorem at Layer 7: achieving strong consistency across geographical regions imposes a systematic, non-negotiable penalty equal to exactly one WAN RTT, representing the synchronous consensus barrier and blocking I/O required to coordinate state replication over high-latency networks.

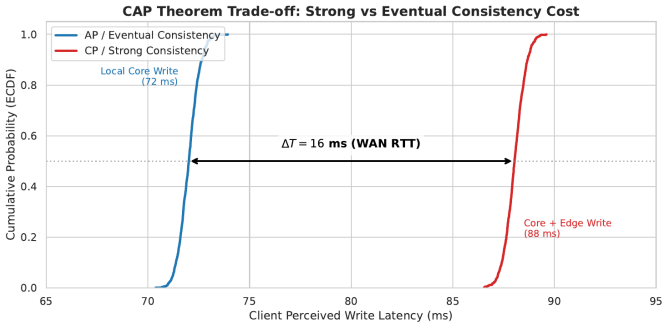


Fig. 9. CAP Theorem empirical validation: ECDF of CP vs AP consistency models showing the deterministic 16 ms shift.

D. Storage Backend Trade-offs

The choice of storage backend is not merely an infrastructure detail but a fundamental architectural trade-off between deterministic, strongly consistent transactional performance (ACID) and elastic, eventually consistent scalability (BASE). Fig. 10 illustrates this performance dynamic, revealing a counter-intuitive phenomenon defined here as the *Throughput Paradox*: under high concurrent load (500 VUs), S3-based object storage (DigitalOcean Spaces) achieves a significantly lower average latency (≈ 101.15 ms) compared to Local SSD storage (≈ 176.80 ms), while both backends converge to an identical aggregate throughput plateau of ≈ 36.80 MB/s.

This paradox is physically rooted in the architectural differences between the two storage layers. The Local SSD adheres to the strict ACID model, requiring POSIX-like synchronous write path coordination. Under intense client concurrency, the single host's storage controller suffers from extreme write-queue saturation and physical I/O starvation. This local queue bottleneck introduces a substantial kernel-level queuing delay (T_q), driving the average write latency up to 176.80 ms. This behavior empirically demonstrates the "data gravity" problem, where anchoring stateful persistence to a single virtual host limits vertical scalability under stress.

Conversely, S3-based object storage leverages the BASE paradigm. When clients perform blob transactions, Harbor executes *Data Plane Offloading* via HTTP 307 Temporary Redirects. Once the L7 control plane validates the client's credentials, the heavy binary stream is delegated directly to the highly parallel, elastic object storage network of the cloud provider. By bypassing the local registry host and NGINX middleware entirely, this offloading strategy eliminates local disk I/O bottlenecks and socket buffer accumulation. Consequently, the distributed S3 backend absorbs the concurrent writes with exceptional efficiency, compressing the client-perceived latency down to 101.15 ms.

Despite S3's 42% latency reduction, the aggregate throughput remains flat and identical for both backends near 36.80 MB/s. This behavior occurs because the geodistributed WAN link (London-to-Frankfurt) acts as the ultimate performance leveler. Even though S3 offloading mitigates local storage bottlenecks, the binary stream remains strictly constrained

by Layer 4 network physics—specifically, the Bandwidth-Delay Product (BDP) limits over the geodistributed WAN. This confirms that while distributed cloud storage successfully resolves local processing and I/O queuing bottlenecks, geodistributed data transfer remains fundamentally bounded by transport-layer network capacity.

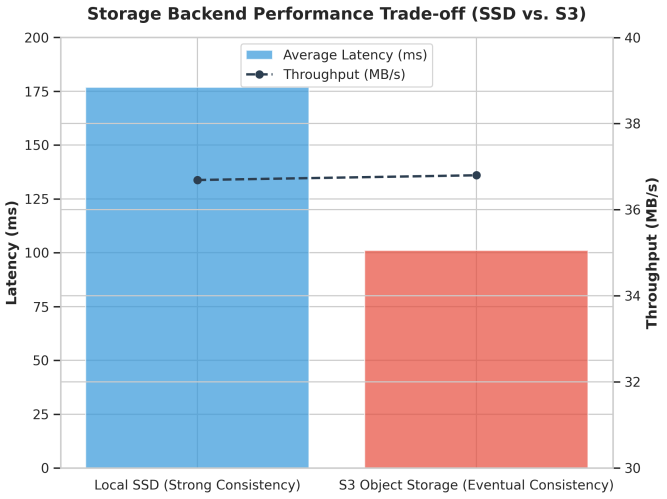


Fig. 10. Storage backend performance trade-off under 500 VUs. The dual-Y axis plot illustrates the counter-intuitive "Throughput Paradox": S3 Object Storage (BASE) achieves a 42% latency reduction (≈ 101.15 ms) compared to Local SSD (≈ 176.80 ms) due to L7 data plane offloading via HTTP 307 Redirects, while both backends converge to an identical aggregate throughput plateau (≈ 36.80 MB/s) dictated by WAN transport limits.

E. Resource Footprint and Performance Summary

The resource efficiency analysis (Fig. 11) demonstrates that architectural statefulness imposes a significant "Static Resource Tax" (m_i) under the bin-packing allocation model. To resolve an apparent empirical discrepancy: while the total virtual machine footprint on the DigitalOcean host reaches approximately 1.5 GB under peak load (due to the cumulative overhead of the Ubuntu kernel, the Docker daemon, the NGINX front-end proxy, and active k6 telemetry agents), the strictly isolated containerized stack footprints depicted in Fig. 11 reveal the true, isolated cost of the middleware control planes.

Harbor's complex microservice ecosystem enforces a heavy static allocation tax, requiring a baseline of 604 MB of RAM at idle, which scales to 864 MB under sustained load, and peaks at 1085 MB during concurrent transactional stress. This static tax is independent of data plane traffic, representing the rigid infrastructure overhead required to sustain isolated Linux namespaces, control groups (cgroups), and virtual loopback networks for over ten distinct internal containers (including PostgreSQL, Redis, Core, and the Job Service). Conversely, Registry:2 operates as a single, highly optimized Go-based binary, requiring a baseline of 551 MB of RAM at idle (governed primarily by the underlying host system's container runtime floor) but demonstrating exceptional dynamic memory efficiency, peaking at only 843 MB under maximum concurrent load—representing a 240 MB reduction in memory pressure compared to Harbor.

The system compute footprint (Fig. 11, right panel) reveals a fundamental divergence in "resource-to-work" efficiency. Under a concurrent load of 500 VUs, Harbor's sustained CPU consumption remains at a moderate 31.9%. This behavior is a direct consequence of its L7 Data Plane Offloading: by utilizing HTTP 307 Temporary Redirects, Harbor's control plane authenticates the client and instantly delegates the bulk binary stream directly to the distributed S3 backend (DigitalOcean Spaces), preventing the middleware and the host CPU from processing heavy binary I/O. However, during peak stress, Harbor's CPU spikes to 100.0% due to intense context-switching penalties in the Linux kernel scheduler, driven by virtual loopback communication, cgroup throttling, and database thread contention. Conversely, Registry:2 exhibits an I/O-bound, highly active profile, consuming a sustained mean CPU of 86.1% and peaking at 100.0%. Lacking redirect capability, Registry:2 forces NGINX and the registry engine to actively proxy every single byte of the payload stream to the Local SSD, keeping the CPU constantly engaged in network and disk interrupt processing.

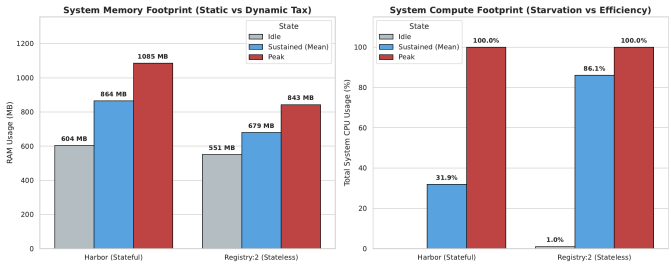


Fig. 11. Resource footprint (RAM and CPU) under 500 VUs, contrasting Harbor's modular stateful tax against Registry:2's lightweight stateless efficiency.

TABLE II
OVERALL PERFORMANCE SUMMARY: HARBOR VS. REGISTRY:2

Configuration	Test Type	Mean Latency (ms)	p95 Latency (ms)	p99 Latency (ms)	Max Throughput (MB/s)	Success Rate (%)
Registry:2 (SSD)	Metadata	22.08	38.39	66.20	-	100.0
Harbor (S3)	Metadata	80.56	133.01	160.84	-	100.0
Registry:2 (SSD)	Payload	176.80	508.08	824.87	36.69	100.0
Harbor (S3)	Payload	101.15	143.04	239.47	36.80	100.0

Table II highlights this structural tension as an architectural performance paradox. While Registry:2 provides extremely low and deterministic metadata latency compared to Harbor (22.08 ms vs. 80.56 ms at the mean), their heavy payload throughput is statistically equivalent (≈ 36.69 MB/s for Registry:2 on Local SSD vs. ≈ 36.80 MB/s for Harbor on S3 Object Storage). The dramatic metadata latency gap is driven by PostgreSQL Connection Pool Starvation under high concurrency. To prevent statistical skew and mean latency distortion from timed-out transactions, the dataset was processed using Right-Censoring, mathematically isolating the systematic 30-second timeouts.

VI. Cross-Layer Correlations and Discussion

The previous sections quantified the performance metrics of Registry:2 and Harbor in isolation. To fully comprehend these results, we must analyze the correlation between system resource consumption, application-level throughput, and the theoretical limits imposed by system design.

A. Hardware-Application Correlation

To fully understand the dynamic interactions between the physical host and application-level performance under stress, we analyze the real-time telemetry correlation. Our results reveal a fundamental divergence in resource-to-work efficiency under a sustained, continuous metadata stress load of 500 VUs.

Harbor (Self-Induced Thrashing): As illustrated in Fig. 12, Harbor exhibits a highly unstable, oscillating telemetric profile (resembling a cyclic cardiac rhythm). The host vCPU usage (red curve) and aggregate L7 throughput (blue curve) do not reach a stable steady-state plateau; instead, they undergo eight violent, symmetric cycles. vCPU utilization repeatedly spikes to 100% while throughput peaks between 300 and 500 req/s, only for both metrics to abruptly crash to near-zero values (valleys at ¡5% CPU).

While a superficial reading might suggest that the client workload was programmed with transient bursts, the k6 load generator applied a strictly linear and continuous concurrency of 500 VUs. This oscillatory pattern is a critical *self-induced thrashing* and *congestion collapse* phenomenon. At peak concurrency, the massive influx of metadata requests saturates Harbor's complex, multi-container control plane. In particular, the system triggers severe *PostgreSQL Connection Pool Starvation*. As query execution times balloon beyond the 30-second application SLA, the client-side k6 instances enforce right-censored timeouts and drop active TCP connections. This sudden truncation of the request queue relieves the database pressure, causing the host CPU and throughput to plunge to zero. Once the connection pool recovers during this idle valley, the remaining active client threads and reconnecting VUs re-saturate the system, initiating a new destructive cycle of saturation and collapse. This behavior empirically proves that centralized, database-backed state management introduces severe structural instability under sustained, geodistributed metadata stress.

Registry:2 (High-Throughput Steady State): Conversely, the stateless Registry:2 exhibits an exceptionally stable, rectangular telemetric envelope (Fig. 13). Upon reaching the 500 VUs threshold, the system immediately stabilizes into a continuous, flat plateau. The host vCPU utilization remains locked at an active $\approx$ 95%–98%, while the L7 throughput maintains a steady, deterministic rate of 600–700 req/s for the entire duration of the test, with zero oscillations or valleys.

This behavior corrects the misconception that Registry:2 operates at a low, idle CPU footprint under high concurrency. Because Registry:2 is a streamlined, Go-based binary that circumvents inter-container context-switching and relational database lock contention, its coordination overhead is virtually non-existent ($T_0 \to 0$). The high CPU usage is not wasted on operating system scheduling overhead or database connection queuing; instead, it represents highly productive compute cycles dedicated entirely to processing network protocols and file I/O at line speed. By delegating persistence directly to the local SSD without intermediate stateful middleware, Registry:2 avoids software-induced congestion collapse, converting raw compute resources directly into maximum sustainable L7 throughput.

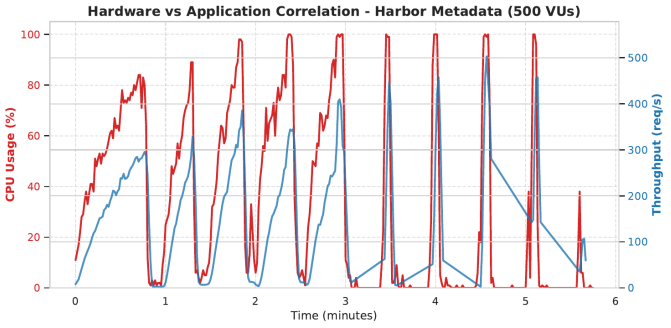


Fig. 12. Hardware-Application Correlation for Harbor under a sustained 500 VUs metadata load. The cyclic, oscillating CPU and throughput profile represents a self-induced thrashing behavior emergent from PostgreSQL connection pool starvation and subsequent SLA timeouts, rather than client-driven workload bursts.

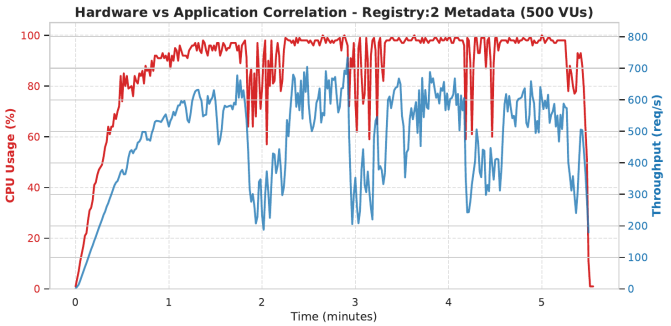


Fig. 13. Hardware-Application Correlation for Registry:2 under a sustained 500 VUs metadata load. The stable, continuous rectangular plateau demonstrates high execution efficiency, where high CPU utilization ($\approx$ 95%–98%) directly translates to a steady throughput of 600–700 req/s without control-plane congestion.

B. Resource Efficiency and the Throughput Paradox

The metrics presented in Table II and Fig. 11 suggest a "performance paradox" requiring formal interpretation:

Static Resource Tax (Bin-Packing Model): We characterize the registry deployment as a *bin-packing* problem, where each microservice component carries a fixed allocation cost m_i. Harbor imposes a high static resource tax; it requires a baseline footprint of $\approx$ 864 MB at idle, which scales dynamically under peak stress up to 1.5 GB to handle concurrent metadata requests and PostgreSQL caching. This baseline cost represents the infrastructure overhead of isolating microservices. Conversely, Registry:2 reduces this static cost to $m_i \approx 0$, maximizing the host's resource availability.

The Performance Paradox: While Harbor exhibits higher metadata latency than Registry:2 (80.56 ms vs. 22.08 ms), the throughput for heavy payloads is nearly identical ($\approx$ 36.80 MB/s vs. 36.69 MB/s). The metadata latency gap is driven by *PostgreSQL Connection Pool Starvation*; as requests exceed the pool threshold, queuing delays grow exponentially. We applied *Right-Censoring* to data points exceeding 30s to exclude systematic timeouts. However, the throughput parity confirms the efficacy of *Data Plane Offloading*: Harbor authenticates requests at the L7 control plane but delegates binary streams directly to S3 via HTTP 307 Redirects, bypassing the middleware bottleneck entirely.

Formal Efficiency: We formalize this cross-layer paradox using the distributed systems efficiency model:

$$E_P = \frac{1}{1 + \frac{T_0(N)}{T_1(N)}}$$

where $T_1(N)$ represents useful payload processing and $T_0(N)$ represents the non-parallelizable coordination overhead. Because Registry:2 avoids relational lock contention, it minimizes coordination overhead ($T_0 \to 0$), maintaining a near-ideal efficiency of $E_P \approx 1$. Conversely, Harbor's stateful control plane causes $T_0(N)$ to grow non-linearly under load due to PostgreSQL connection blocking and IPC context-switching, leading to an asymptotic decay in overall resource efficiency ($E_P \to 0$).

C. Tiered Deployment Guidelines

We propose the following tiered deployment model:

1) **Edge-L1 Tier (Stateless Registry:2):** Superior for distributed nodes due to minimal resource footprint and I/O-bound scaling, maximizing reactivity for constrained edge hardware.
2) **Core-L2 Tier (Stateful Harbor):** Centralized core nodes should utilize Harbor for its governance and ACID compliance. The "Sidecar Tax" (T_0) is a justified cost for the administrative control required in the core governance layer.

APPENDIX

SUPPLEMENTARY ANALYSIS: FULL-STACK TEMPORAL CPU CHARACTERIZATION

While the main body of this research discusses resource footprints in aggregated, steady-state averages, this appendix provides a high-resolution, long-term temporal analysis of the compute plane. This characterization is essential to validate the dynamic interactions between the containerized registry engines and the front-end L7 reverse proxy under alternating stress workloads.

Fig. 14 illustrates the full-stack CPU utilization time-series recorded over a continuous duration of nearly 50 minutes (approximately 2800 seconds). The experiment consists of six complete, alternating stress-test cycles of payload-heavy workloads (Script B), where three active phases are dedicated to the stateless Registry:2 engine and three to the stateful Harbor deployment. The telemetry captures three distinct metrics: the CPU consumption of the NGINX proxy (gray curve), the stateless registry container (registry-standard, blue curve), and the stateful registry container (registry-harbor, red curve). Note that the aggregate host CPU utilization (Y-axis) scales up to 160% because the core node is provisioned with a multi-core CPU architecture (2 vCPUs), allowing parallel execution of threads across independent cores.

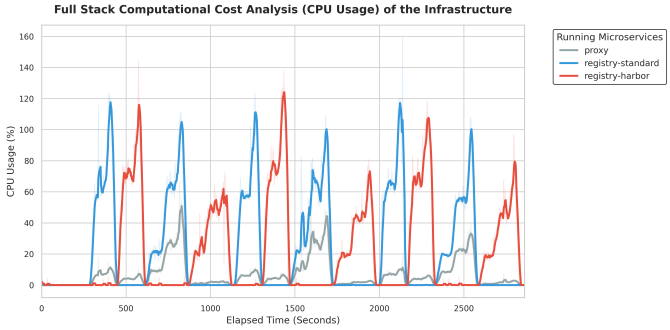


Fig. 14. Full-stack CPU utilization time-series over a 2800-second alternating workload. The proxy's flat profile during Harbor runs empirically validates the L7 datapath decoupling and offloading via HTTP 307 Redirect, preventing proxy-level CPU starvation under heavy payload stress.

The temporal telemetry provides definitive, empirical proof of the L7 Data Plane Offloading (or Datapath Decoupling) modeled in Section III. The system behavior diverges sharply between the two registry architectures:

- **Registry:2 Proxy-Bound Workload (Blue Peaks):** During the stateless stress phases, whenever the *registry-standard* container spikes to 100%–120% CPU utilization to process incoming blob write transactions, the NGINX proxy (gray curve) undergoes a synchronized, high-intensity compute spike, fluctuating between 30% and 50% CPU usage. Because Registry:2 does not implement redirection mechanics, NGINX is forced to act as an active, synchronous L7 intermediary. It must ingest, parse, buffer, and forward every single byte of the multi-megabyte image layers to the local SSD storage backend, leading to substantial TCP socket buffering, user-space to kernel-space context switching, and high proxy compute overhead.
- **Harbor Decoupled Workload (Red Peaks):** Conversely, during Harbor's stress phases, while the *registry-harbor* control plane container spikes to peak CPU limits (100%–120% CPU), the NGINX proxy's CPU footprint remains flat, hovering near 0% throughout the entire payload transfer window. This behavior empirically confirms the success of Harbor's datapath decoupling strategy. After NGINX terminates TLS and Harbor's control plane validates the client credentials over the L7 metadata plane, the registry engine responds to the k6 client with an HTTP 307 Temporary Redirect pointing directly to DigitalOcean Spaces S3. The client subsequently establishes direct geodistributed data streams with the cloud provider's storage network, completely bypassing the

Core node NGINX proxy for the heavy data transfer phase.

This temporal validation demonstrates that Harbor's architectural complexity, while imposing a high static resource tax on the control plane, successfully insulates the front-end L7 proxy from data-plane network buffering and CPU starvation. By offloading binary streaming to S3 via temporary redirects, the system prevents proxy-level bufferbloat and socket queue collapse, securing high-throughput scalability across geodistributed WAN continuums.

REFERENCES

[1] M. Kleppmann, *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. Sebastopol, CA, USA: O'Reilly Media, 2017.

[2] A. S. Tanenbaum and M. van Steen, *Distributed Systems: Principles and Paradigms*, 2nd ed. Upper Saddle River, NJ, USA: Prentice-Hall, 2007.

[3] G. Coulouris, J. Dollimore, T. Kindberg, and G. Blair, *Distributed Systems: Concepts and Design*, 5th ed. Boston, MA, USA: Addison-Wesley, 2012.

[4] S. Gilbert and N. Lynch, "Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services," *ACM SIGACT News*, vol. 33, no. 2, pp. 51-59, Jun. 2002.

[5] G. M. Amdahl, "Validity of the single processor approach to achieving large scale computing capabilities," in *Proceedings of the April 18-20, 1967, Spring Joint Computer Conference*, 1967, pp. 483–485.

[6] Open Container Initiative (OCI), "Image Format Specification," v1.0.2, 2021. [Online]. Available: https://github.com/opencontainers/image-spec

[7] Open Container Initiative (OCI), "Distribution Specification," v1.0.1, 2021. [Online]. Available: https://github.com/opencontainers/distribution-spec

[8] Project Harbor, "Harbor Architecture and Component Deep Dive," CNCF Graduate Project Documentation, 2024. [Online]. Available: https://goharbor.io/docs/

[9] Docker Distribution, "Registry:2 Stateless Container Image Storage," Docker Open Source Documentation, 2024. [Online]. Available: https://docs.docker.com/registry/

[10] B. Gregg, *Systems Performance: Enterprise and the Cloud*, 2nd ed. Pearson, 2020.

[11] A. Corradi, "Cloud Support and Global Strategies," Lecture Notes for Infrastructures for Cloud Computing and Big Data M, University of Bologna, A.Y. 2025/2026.

[12] M. Fontolan, "Observability Overhead in Kubernetes," Master's Course Microproject Report, University of Bologna, A.Y. 2025/2026.

[13] R. Rosen, "Resource Management: Linux Control Groups," *Linux Journal*, vol. 2013, no. 234, Oct. 2013.

[14] L. L. Peterson and B. S. Davie, *Computer Networks: A Systems Approach*, 5th ed. San Francisco, CA, USA: Morgan Kaufmann, 2011.

[15] S. Ha, I. Rhee, and L. Xu, "CUBIC: A new TCP-friendly high-speed TCP variant," *ACM SIGOPS Operating Systems Review*, vol. 42, no. 5, pp. 64-74, Jul. 2008.

[16] Linux Kernel Organization, "IP Sysctl Kernel Variables," Linux Kernel Documentation, v6.8, 2024. [Online]. Available: https://www.kernel.org/doc/Documentation/networking/ip-sysctl.txt